
Database Integration Lab (Practical)

Welcome to the practical lab for Database Integration! In this session, you will connect your FastAPI backend to a local **MySQL** database using **SQLAlchemy 2.0**, define database models using Python classes, configure session management dependencies, and save your machine learning prediction logs directly into the database.

1. Concept Overview

In this lab, you will:

- Install MySQL drivers (`PyMySQL`) and SQLAlchemy for Python.
 - Establish a database connection string and configure session management.
 - Define an ORM model (`PredictionModel`) that automatically creates a table in MySQL.
 - Use FastAPI dependency injection (`Depends`) to open and close database sessions safely per request.
-

2. Step-by-Step Lab Instructions

Step 1: Install Required Database Packages

Activate your virtual environment in your terminal and install SQLAlchemy and PyMySQL (the pure-Python MySQL client):

```
pip install sqlalchemy pymysql cryptography
```

Step 2: Configure Database Connection (`database.py`)

Create a new file named `database.py` in your project root to handle the database engine and session factory:

```
# database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker
```

```

# Replace 'root' and 'your_password' with your local MySQL credentials
# Format: mysql+pymysql://username:password@host:port/database_name
DATABASE_URL =
"mysql+pymysql://root:your_password@localhost:3306/ml_backend_db"

# Create the SQLAlchemy engine
engine = create_engine(DATABASE_URL)

# Create a configured "SessionLocal" class
SessionLocal = sessionmaker(autocommit=False, autoflush=False,
bind=engine)

# Base class for our declarative models
class Base(DeclarativeBase):
    pass

# Dependency function to get a database session per request
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

Note: Make sure you create the database schema in MySQL Workbench first by running: `CREATE DATABASE ml_backend_db;`

Step 3: Define SQLAlchemy Model (`models.py`)

Create a new file named `models.py` to define the database schema using SQLAlchemy 2.0 syntax:

```

# models.py
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import Float, DateTime, func
from database import Base
import datetime

class PredictionRecord(Base):
    __tablename__ = "predictions"

    id: Mapped[int] = mapped_column(primary_key=True, autoincrement=True)
    feature1: Mapped[float] = mapped_column(Float, nullable=False)
    feature2: Mapped[float] = mapped_column(Float, nullable=False)
    prediction_result: Mapped[float] = mapped_column(Float,
nullable=False)
    created_at: Mapped[datetime.datetime] = mapped_column(DateTime,
server_default=func.now())

```

Step 4: Integrate Database with FastAPI (`main.py`)

Update your `main.py` file to initialize the database tables, accept prediction inputs, save them to MySQL, and return the stored records:

```
# main.py
from fastapi import FastAPI, Depends, HTTPException
from pydantic import BaseModel
from sqlalchemy.orm import Session
from database import engine, Base, get_db
from models import PredictionRecord

# Automatically create database tables if they don't exist
Base.metadata.create_all(bind=engine)

app = FastAPI(title="ML Backend with MySQL", version="1.0")

class PredictionInput(BaseModel):
    feature1: float
    feature2: float

@app.get("/")
def read_root():
    return {"message": "FastAPI and MySQL integration active!"}

@app.post("/predict")
def create_prediction(data: PredictionInput, db: Session = Depends(get_db)):
    # Calculate mock prediction score
    score = (data.feature1 + data.feature2) * 1.5

    try:
        # Create a new database record instance
        db_record = PredictionRecord(
            feature1=data.feature1,
            feature2=data.feature2,
            prediction_result=round(score, 2)
        )

        # Add to session, commit transaction, and refresh
        db.add(db_record)
        db.commit()
        db.refresh(db_record)

        return {
            "status": "success",
            "record_id": db_record.id,
            "prediction": db_record.prediction_result,
            "created_at": db_record.created_at
        }
    except Exception as e:
        db.rollback()
        raise HTTPException(status_code=500, detail=str(e))
```

```
@app.get("/predictions")
def get_all_predictions(db: Session = Depends(get_db)):
    # Query all records from MySQL table
    records = db.query(PredictionRecord).all()
    return {"total_records": len(records), "data": records}
```

Step 5: Run and Test Your API

1. Start your FastAPI server:

```
uvicorn main:app --reload
```

2. Open **Postman** or navigate to Swagger UI (`[http://127.0.0.1:8000/docs]` (`http://127.0.0.1:8000/docs`)).
3. Send a **POST** request to `/predict` with JSON features. Check MySQL Workbench (`SELECT * FROM ml_backend_db.predictions;`) to verify that the record was successfully saved!
4. Send a **GET** request to `/predictions` to retrieve all saved prediction logs.

3. Reference Links & Further Reading

- **SQLAlchemy Session Basics:** Guide on managing session life cycles and transactions.
 - **FastAPI SQL Databases Tutorial:** Official guide on structuring FastAPI apps with SQLAlchemy.
-